

# SAP ABAP Classic REST API 課程

OOP 課程 ( [src/ABAP\\_Training\\_OOP/](#) · op01–op13 ) 的續篇。課綱草案 ( 2026-07-11 ) · 尚未出題。

## 課程定位

- 對象：完成 OOP 課程的學員 ( 會 Class/Interface/例外/ABAP Unit/cl\_salv\_table )
- 技術範圍：Classic REST ( [CL\\_REST\\_HTTP\\_HANDLER](#) + [CL\\_REST\\_RESOURCE](#) ) · 非 RAP/CDS 那條線的現代 REST ( 那是另一階段候選主題 )
- 結業標準：能獨立設計一個小型 CRUD REST Service——Application/Resource 分工、URI 路由、JSON 序列化、狀態碼語意、統一錯誤格式、商業邏輯與 REST 轉接層分離並附 ABAP Unit 測試
- 本課程題號 = 授課順序 · rs09 ( 期末整合 ) 是原訂課綱最後一題；rs10/rs11 是 2026-07-13 驗收 rs09 後追加的延伸題 ( 使用者要求補上「JSON Array → Internal Table → CALL BAPI」「Header/Item 單一 BAPI 呼叫」這兩個原課綱沒有的模式 · 不算在原本「結業」範圍內，但屬於同一份教材 )；rs11 為了示範「Header(1)+Item(多筆) 一次 BAPI 呼叫」跳出 SFLIGHT/SBOOK 主題改用銷售訂單模型 · 原因見該題 md 開頭說明

## 教材慣例

- 每題三件套：題目 [rsNN\\_主題.md](#) + PDF 講義 ( [node tools/md2pdf.js src/ABAP\\_Training\\_REST](#) ) + 答案快照
- 每題 md 開頭 ( [## 學習目標](#) 之前 ) 都要有一段 [## Lecture](#)：完整的背景知識/概念講解 ( 這題所屬的 REST 概念是什麼、為什麼需要它、跟前後幾題的關聯 ) · 讓每題本身就是一份小講義 · 不是只有「題目 + 答案」——這是 2026-07-12 出到 rs06 之後使用者要求補上的慣例 · rs01~rs06 已回頭補齊 · rs07 起新題一律先寫這段再寫學習目標
- 答案物件命名：[ZCL\\_RSnn\\_\\*](#) ( Application / Resource / Service 類別 ) / [ZCX\\_RSnn\\_\\*](#) ( 例外類別 ) · 套件 [\\$TMP](#)
- 資料模型：沿用 SCARR/SFLIGHT/SBOOK 航班訓練模型 · 與 OOP 課程一致 · 不額外建 DDIC 表
- SICF 節點掛載無 ADT API ( 跟 T-code、Search Help 一樣是 GUI-only · 見 [.claude/rules/sap-adt-mcp.md](#) 第 9/12 節 ) · 每題若要在瀏覽器/Postman 實測 · SICF 手動掛載步驟寫在題目 md 裡 · 由學員 ( 你 ) 在 SAP GUI 操作；Claude 負責 Handler/Resource/Service 類別的程式碼與 ADT 端啟用
- SPROX\_HTTP\_REQUEST 測試一律要填完整網址 ( [http://<主機>:<port>/sap/bc/...](#) ) · 不能只填 [/sap/bc/...](#) 相對路徑：這支程式雖然是在 Application Server 上跑、位於內網 · 但執行的是一支真正發出去的 HTTP request ( 呼叫 ICM 的 HTTP Port ) · 不是程式內部呼叫 · 缺 host:port 會找不到 API endpoint；每題 md 的 SICF 測試步驟都要用 [http://<主機>:<port>/...](#) 這種完整格式 · 不要只寫路徑 ( 2026-07-12 曾在 rs04/rs05 漏寫 · 已修正 )

## 課綱 ( 草案 · 待逐題出題與驗收 )

| #    | 主題                                      | 內容重點                                                                                                                                                                                     | 銜接 OOP 課程                                            | 狀態  |
|------|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|-----|
| rs01 | 為什麼要 REST + 架構總覽                        | HTTP 動詞語意 ( GET/POST/PUT/DELETE 與冪等性 ) · <a href="#">CL_REST_HTTP_HANDLER</a> / <a href="#">CL_REST_RESOURCE</a> / SICF 三者關係、最簡單的純文字 GET echo service                                    | 對照 op01「為什麼要 OOP」的破題方式                               | 已驗收 |
| rs02 | 第一個 REST Service：Application 與 Resource | <a href="#">IF_REST_APPLICATION~GET_ROOT_HANDLER</a> · <a href="#">CL_REST_ROUTER~ATTACH</a> 、Resource Class 繼承 <a href="#">CL_REST_RESOURCE</a> 覆寫 <a href="#">IF_REST_RESOURCE~GET</a> | 承 op05 全域類別、op07 介面                                  | 已驗收 |
| rs03 | JSON 序列化與集合查詢                           | <a href="#">/ui2/cl_json</a> · SFLIGHT 查詢結果轉 JSON array、Content-Type 設定                                                                                                                  | 對照 op11 <a href="#">cl_salv_table</a> 的「標準 API 實戰」定位 | 已驗收 |
| rs04 | URI 路徑參數與單筆查詢                           | <a href="#">GET_URI_ATTRIBUTE</a> 取路徑變數 ( <a href="#">/flights/{carrid}/{connid}/{fldate}</a> · 帶完整主鍵才能唯一定位 ) · 集合 vs 單筆兩種 GET 邏輯、查無資料回 404                                              | —                                                    | 已驗收 |

| #               | 主題                                | 內容重點                                                                                                                                                                                                                             | 銜接 OOP 課程                                                   | 狀態                                                                                                                                     |
|-----------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| rs05            | Query Parameter 篩選                | HAS_URI_QUERY_PARAMETER/GET_URI_QUERY_PARAMETER、篩選條件用布林旗標 OR 動態組合 WHERE ( <b>carrid/connid/fromdate</b> )、篩選值本身不存在回 404 vs 合法但空結果回 200+[]                                                                                        | 承 op09 例外處理的錯誤情境設計                                          | 已驗收                                                                                                                                    |
| rs06            | POST 建立資源                         | IO_ENTITY->GET_STRING_DATA 讀 body + JSON 反序列化、必填欄位與外鍵驗證回 400、主鍵重複回 409、201<br><b>Created</b> + Location Header、 <b>CSRF</b> 保護：覆寫 <b>HANDLE_CSRF_TOKEN</b> 跳過檢查 ( 僅供本課程測試工具使用，說明適用時機 )                                           | —                                                           | 已驗收                                                                                                                                    |
| rs07            | PUT/DELETE 與統一錯誤格式                | PUT 更新、DELETE 刪除、狀態碼語意 ( 200/204/400/404 )、 <b>ZCX_</b> 例外攔截轉標準 JSON error body，避免未捕捉例外變成 dump                                                                                                                                   | 承 op09 自訂例外類別設計                                             | 已驗收<br>( GET/PUT/DELETE 全部已測；DELETE 因 <b>SPROX_HTTP_REQUEST</b> 無此選項，改用使用者自建的 <b>ZTEST_HTTP_DELETE</b> 程式測試，204 + 後續 GET 回 404 皆符合預期 ) |
| rs08            | REST Resource 的可測試性               | 商業邏輯搬進 <b>ZCL_RSnn_*_SERVICE</b> ，Resource 只做「解析 request → 呼叫 Service → 組 response」的薄層轉接，Service Class 附 ABAP Unit 測試                                                                                                            | 兌現 op10 ABAP Unit + op12 重構精神                               | 已驗收 ( ABAP Unit 6/6 綠燈 + SICF 手動測試 )                                                                                                   |
| rs09            | 期末整合：Flight Booking CRUD REST API | SBOOK 的完整 GET ( 集合+單筆+篩選 ) /POST/PUT/DELETE，Service/Resource 分層、統一錯誤格式、ABAP Unit 測試，SICF 掛載後用 Postman/瀏覽器實測全流程                                                                                                                   | 對照 op12 期末報表重構的收斂角色                                         | 已驗收 ( ABAP Unit 8/8 綠燈；SICF 手動測試通過 )                                                                                                   |
| rs10<br>( 延伸題 ) | JSON Array 批次處理與 BAPI 整合          | POST 收 JSON Array、 <b>/UI2/CL_JSON</b> 直接反序列化進 Internal Table、 <b>LOOP</b> 呼叫標準 BAPI <b>BAPI_FLBOOKING_CREATEFROMDATA</b> 批次建立訂位、逐筆成功/失敗回報 ( 非全有全無 )、 <b>BAPI_TRANSACTION_COMMIT</b>                                             | 補課綱原本沒有的「Header/Detail、多筆輸入、呼叫 BAPI 更新表格」缺口                 | 已驗收 ( ABAP Unit 2/2 綠燈；SICF 手動測試通過，過程中發現並修正 <b>counter</b> 欄位缺口 )                                                                      |
| rs11<br>( 延伸題 ) | Header/Item 單一 BAPI 呼叫 ( 銷售訂單模型 ) | 跳出 SFLIGHT/SBOOK，改用 <b>BAPI_SALESORDER_CREATEFROMDAT2</b> ：Header(1)+Item(多筆) 一次呼叫、 <b>XXX_IN/XXX_INX</b> 成對 X 旗標結構、 <b>items</b> 巢狀在 <b>header</b> 底下的兩層 JSON 反序列化 ( 這門課第一次 )、把 <b>TESTRUN</b> 暴露成 <b>?testrun=true</b> 查詢參數供預覽 | 補課綱原本沒有的「一次 BAPI 呼叫處理 1 對多」缺口 ( 跟 rs10「迴圈呼叫單筆 BAPI N 次」互補 ) | 已驗收 ( ABAP Unit 6/6 綠燈；SICF <b>testrun=true</b> 預覽與正式建立皆通過， <b>VBAK/VBAP</b> 確認兩筆品項寫入成功 )                                              |

## 不碰的範圍 ( 下一階段 )

RAP ( RESTful ABAP Programming Model ) / CDS 那條現代化路線、OAuth 等正式驗證機制 ( 僅在深度選項提到但本輪課綱未收錄 )、Design Pattern。

## 出題工作流程 ( 比照 OOP 課程 )

SAP 建物件 ( **sap\_create\_object**；MCP 不支援的物件類型見 [.claude/rules/sap-adt-mcp.md](#) 的 ADT API workaround ) → **sap\_set\_source** 寫入 → curl 啟用 + 語法檢查 → 若該題需要 SICF 才能實測，題目 md 附手動掛載步驟由使用者操作 → 使用者驗收 → 快照 ( curl 下載 active 版本 ) → 題目 md → **node tools/md2pdf.js src/ABAP\_Training\_REST** 產 PDF → 更新本 README 狀態 → commit。每批 2-3 題、使用者驗收後再繼續。